# Proiect SPG
# Simularea unei masinii de politie

Student: Rarinca Robert-Mario
Grupa: 1306A

## I. Introducere

Proiectul prezentat a avut ca tema simularea unei masini de politie, intr-un oras, in timpul noptii. Timpul simularii (noaptea) a fost aleasa pentru a demostra modelele de luminare si umbrire posibile in OpenGL, in special modelul de luminare si umbrire phong. Masina de politie a fost aleasa datorita semnalelor luminoase speciale. Tema de oras a fost aleasa datorita posibilitatii surprinderii efectelor de luminare si umbrire pe cladiri.

Tema generala a proiectului a fost aleasa dupa ce un coleg a propus sa fac "Need For Speed: Most Wanted (2005)" (desi nu cred ca am atins acelasi nivel). De la aceasta idee a fost implementata si masina 'criminala'.

De asememea, tema a fost inspirata de jocurile cu o tema de stil 'arcade', de aici vine camera foarta indepartata de jucator la viteza maxima si inclinarile exagerate cand se ia un viraj la o viteza mare.

Printre obiectivele propuse se numara: crearea masinii de politie, crearea orasului, implementarea modelului phong de luminare si umbrire, implementarea technicilor de "shadow map" si "shadow cube" pentru crearea de umbre, urmarirea unei dezvoltarii software ordonata bazata pe principiile programarii orientate obiect, precum si respectarea standarelor de scriere programelor C++ (Resource Aqcuisition Is Initialization – RAII, Rule of Five, Rule of Zero, Minizare RunTime Type Information - RTTI, si altele).

## II. Metodologie

La nivel de granulate mare, pentru dezvoltarea programului de simulare, au fost folosite GLAD, OpenGL 4.6, GLFW, Visual Studio 2022, compilatorul MSVC (Microsoft Visual C), C++20. Proiectul include de asemenea STB_Image.

La nivel de granularitate medie, s-au folosit tehnici de scriere a unui program orientat obiect (incapsulare, abstractizare, mostenire, polimorfism),

precum si tehnici de scriere in limbajul C++ (Rule of Zero / Three / Five, minimizare RTTI, RAII, etc). S-a optat pentru o structurare a proiectului de tipul “One Class, One File” (O clasa, Un fisier); de fapt este “One Class, Two Files”.

La nivel de granulatite mica, s-au folositi algoritmi consacrati pentru modelul de iluminare si umbrire phong, dar si pentru technicile de umbrire de tip “shadow map” si “shadow cube”, cu mici modificari. Pentru crearea orasului, sa folosit o technica de “Jittered Grid”, si “Manhattan Routing”. Pentru ‘masina criminala’ a fost folosit un algoritm Greedy global, si algoritmul BFS (Breadth First Search). Pentru efectul de ‘x-ray’ al masinii, s-a folosit un alt shader unde s-au setat flag-urile necesare pentru a fi randat ‘deasupra’ celorlate obiecte. Transparenta geamurilor masinilor a fost facuta prin setarea texturii ferestrelor la o textura de 1x1 pixeli cu o culoare si opacitate data.

GLAD este un “loader-generator” multi-limbaj pentru mai multe librarii grafice (Vulkan, OpenGL, etc). Pentru acest proiect sa folosit pentru a genera un “loader” pentru OpenGL 4.6 in modul compatibilitate.

OpenGL (Open Graphics Library) este o biblioteca multi-platforma folosit pentru randarea de obiecte 2D si 3D, folosit can un ‘pod’ standardizat intre software si hardware, permitand trimiterea de comenzi direct la GPU.

GLFW este o biblioteca multi-platforma pentru OpenGL pentru afisare, creare si manipulare de contexte (ferestre) grafice. Acest modul este responsabil pentru desenare si interactiunea cu utilizatorul (preluarea de apasari de taste, preluare miscare mouse, si altele).

Visual Studio 2022 este un “Integrated Development Environment” (prescurtat IDE), care include unelte de compilare MSVC (Microsoft Visual C / C++), precum si unelte de depanare al codului scris la rulare. Acesta include elemente grafice pentru a usura dezvoltarea, crearea si depanarea de software.

Standardul oficial (la momentul scrierii) se numeste C++20, si este un document public de ISO (International Standardization Organisation) ce defineste reguli, sintaxa si caracteristici ale limbajului C++. Acest standard este pus la dispozitie, in special pentru dezvoltatorii de compilatoare, pentru a se asigura ca codul sursa C++ produce acelasi cod executabil indiferent de compilatorul si hardware-ul folosit.

STB_Image este o biblioteca open-source (cu codul sursa pus la dispozitie) pentru incarcarea de imagini intr-un mod mult mai facil intr-un cod scris in

limbajul C / C++. Desi STB este o colectie de astfel de librarii, pentru acest proiect s-a folosit doar libraria de incarcarea de imagini.

Programarea orientata obiect are la baza obiecetele, si 4 reguli ce definesc manipularea acestor obiecte. Incapsularea reprezinta ascunderea starii interne a acestor obiecte, si expunerea (sau nu) si manipularea a acestor stari doar prin metode publice. Abstractizarea reprezinta preluarea doar a atributelor necesare si folositoare ale unui obiect cand acest este proiectat si implementat, dar si ascunderea detaliilor de implementare a functionalitatilor interne. Mostenirea este folosita pentru reutilizarea de cod si crearea de ierarhii de clase. Polimorfismul reprezinta comportamentul diferit al unor obiecte desi au aceeasi interfata (de exemplu, o metoda draw() are un comportament diferit pentru un patrat si un cerc, desi ambele sunt figuri geometrice).

In limbajul de programare C++, Rule of Zero / Three / Five este folosit in cazul in care o clasa (nu) gestioneaza resurse 'critice'. De obicei aceste resurse critice sunt pointeri 'raw' (tip* nume), handler-uri (cum ar fi VBO, EBO, VAO, texture IDs din OpenGL). Rule of Zero este pentru clase ce nu au aceste resurse critice, in special pentru evitarea folosirea acestora fara un motiv intemeiat (cum ar fi pointer la char in loc de std::string). Clasele ce respecta Rule of Zero de obicei au un destructor "default" (generat automat de catre compilator). Rule of Three / Five sunt pentru clasele ce au aceste resurse critice, si dupa regula data, ar trebui sa defineasca, pe langa destructor, si constructorul de copiere si operatorul de asignare copiere. Rule of Five este o extindere a Rule of Three, prin definirea constructorului de mutare si operatorului de asignare mutare. Aceste reguli sunt folositoare pentru a evita bug-urile produse de generarea automata a acestor constructori, operatori si destructor de catre compilator, ceea ce poate duce la pierderi de memorie, fenomenul de "double free" (incercarea elibarii unui pointer care a fost deja eliberat) si alte erori la rulare.

O clasa ce urmeaza Rule of Five, ce nu foloseste pointer-i, dar handler-uri (campul ID) pentru texturi OpenGL, este urmatoarea:

```
class Texture
{
private:
    unsigned int ID = 0;
    Texture(const Texture&) = delete;
    Texture& operator=(const Texture&) = delete;
```

```
  public:
          Texture(const std::string& path, const
std::string& uniformName);
      Texture(uint8_t* data, int width, int height,
int components, const std::string& uniformName);
      Texture(Texture&& other) noexcept;
      ~Texture();
      // Alte metode specifice.
      Texture& operator=(Texture&& other) noexcept;
  };
  Texture& Texture::operator=(Texture&& other) noexcept
  {
      if (this == &other)
      {
          return *this;
      }
      glDeleteTextures(1, &ID);
      ID = std::exchange(other.ID, 0);
      uniformName = std::move(other.uniformName);
      return *this;
  }
  Texture::Texture(Texture&& other) noexcept :
      ID(std::exchange(other.ID, 0)),
      uniformName(std::move(other.uniformName)) {}
  Texture::~Texture() { glDeleteTextures(1, &ID); }
```

RAII (Resource Acqusition Is Initialization) este o paradigma specifica limbajului de programare C++. Acesta stipuleaza ca achizitionarea de resurce (de obicei acelease ‘resurse critice’ ca mai sus; pointeri, handlers, etc) necesita instantierea unui obiect, de obicei de tip ‘wrapper’, peste aceste resurse. Acesta face ca durata de viata acestui obiect, si implicit a acestor resurse critice

alocate, sa fie strans legate. In acest sens, de obicei se spune ca initializarea obiectului aloca resursa, si distrugerea obiectului elibereaza resursa.

Un exemplu de RAII ar fi urmatorul (clasa Shader este un wrapper peste shader-urile specifice OpenGL)

```
      auto phong =
std::make_shared<Shader>("phong.vert", "phong.frag");

      shaders[static_cast<size_t>(ShaderType::Phong)] =
phong;

      auto depth =
std::make_shared<Shader>("shadow_map.vert",
"shadow_map.frag");

    shaders[static_cast<size_t>(ShaderType::ShadowMap)]
= std::move(depth);

      auto cube =
std::make_shared<Shader>("shadow_cube.vert",
"shadow_cube.frag", "shadow_cube.geom");

   shaders[static_cast<size_t>(ShaderType::ShadowCube)]
= std::move(cube);


      auto xray = std::make_shared<Shader>("xray.vert",
"xray.frag");

      shaders[static_cast<size_t>(ShaderType::Xray)] =
std::move(xray);
```

RTTI (RunTime Type Information) este o caracteristica a limbajului de programare C++. Aceasta este folosite pentru a prelua tipul unei variabile (de obicei o clasa) in timpul rularii, pentru a face cast, sau operatii specifice pe acea variabila. In cele mai multe ghiduri de scriere a unui program C++, se recomanda a nu se folosi acest mecanism, si inlocuirea sa cu alte metode de apelare acestor functionalitati (sablonul Visitor, re-proiectarea ierarhiei de clase pentru a tine cont de aceasta functionalitate, si altele).

Structurarea proiectului in modul “One Class, One File” (de fapt, “One Class, Two Files”) inseamna, dupa cum spune si numele, ca fiecare clasa este definita intr-un singur fisier (de obicei cu extensia .h sau .hpp), iar implementarea propriu zisa intr-un alt fisier (de obicei cu extensia .c sau .cpp). Denumirea de “One Class, One File” este mult mai raspandita decat “One Class, Two Files”, desi in acest caz se folosesc doua fisiere pentru definire si

implementare. Acest mod de structurare este in contrast cu stilul “vechi” de structurare monolit (un singur fisier .h cu toate declaratiile, clasele, etc, si un singur fisier .cpp cu implementarile).

Modelul de iluminare si umbrire phong este caracterizat de trei tipuri de iluminare, cea ambienta (constanta), difuza (lumina ce loveste obiectul la un unghi) si speculara (partea stralucitoare a unui obiect). Aceste se calculeaza preluand distanta de la obiect la sursa de lumina (mai putin cea ambienta, care este constanta indiferent de distanta), si folosind aceastea, calcularea de coeficienti pentru o anumita lumina in scena cu o putere de iluminare data, ceea ce va face ca obiectul sa para iluminat intr-un mod natural de catre sursa de lumina.

In shader-ul de fragmente al modelului de iluminare phong, se defineste urmatoarea bucla pentru tratarea luminii provenite de la ‘point lights’ (lumini omnidirectionale):

```
  for (int i = 0; i < pointLightCount; i++)
  {
      if(pointLights[i].intensity <= 0.001f)
      {
          continue;
      }

      float dist = length(pointLights[i].position -
FragPos);
      if (dist > 30.0)
      {
          continue;
      }
      float currentFarPlane = (i < 2) ?
lightBarFarPlane : blinkerFarPlane;
      float window = 1.0;
      float farFade = clamp((currentFarPlane - dist) /
window, 0.0, 1.0);
```

```
      float attenuation = (1.0 /
(pointLights[i].constant + pointLights[i].linear * dist
+ pointLights[i].quadratic * (dist * dist))) * farFade;

      if (attenuation < 0.001)

      {

          continue;

      }

      vec3 lightDir = normalize(pointLights[i].position
- FragPos);

      float NdotL = max(dot(norm, lightDir), 0.0);

      if (NdotL <= 0.0)

      {

          continue;

      }

      float shadow = 0.0;

       vec3 reflectDir = reflect(-lightDir, norm);

      float spec = pow(max(dot(viewDir, reflectDir),
0.0), shininess);

      vec3 ambient = (ambientStrength * 0.05) *
pointLights[i].color;

      vec3 diffuse = NdotL * pointLights[i].color *
pointLights[i].intensity;

      vec3 specular = specularStrength * spec *
pointLights[i].color * pointLights[i].intensity;

      result += (ambient + (1.0 - shadow) * (diffuse +
specular)) * attenuation;

  }
```

Metoda de umbrire de tipul “shadow mapping” foloseste metoda de ‘depth testing’ din perspectiva luminii (~ camera priveste in directia luminii). Aceasta randeaza scena din perspectiva luminii si calculeaza distanta (‘depth’) a unui pixel in prima parte, iar in randarea principala (obiecte, culori, etc), pozitia aceluiasi pixel este comparata cu ‘depth map’-ul (textura) umbrei. Daca pixel-ul se afla ‘mai departe’ de pozitia pixelului in textura umbreii, acesta este

ascuns si desenat ca o umbra. Acesta tehnica este potriivata pentru lumini de tip reflector (ce ilumineaza intr-o singura directie).

Pentru calcularea umbrei date de un shadow map, sa folosit urmatoarea:

```
  float ShadowCalculation(sampler2D shadowMap, vec4
fragPosLightSpace, float NdotL, vec3 normal)

  {

      vec3 projCoords = fragPosLightSpace.xyz /
fragPosLightSpace.w;

      projCoords = projCoords * 0.5 + 0.5; // [-1,1] ->
[0,1]

      if(projCoords.z > 1.0)

      {

          return 0.0;

      }

      if(projCoords.x < 0.0 || projCoords.x > 1.0 ||
projCoords.y < 0.0 || projCoords.y > 1.0)

      {

          return 0.0;

      }

      float offsetScale = 0.05;

      vec2 offset = normal.xz * (1.0 - NdotL) *
offsetScale * (1.0 / textureSize(shadowMap, 0));

          float bias = max(0.0003 * (1.0 - NdotL),
0.0003);

      float currentDepth = projCoords.z;

      vec2 texelSize = 1.0 / textureSize(shadowMap, 0);

      float shadow = 0.0;

      for(int x = -1; x <= 1; ++x)

      {

          for(int y = -1; y <= 1; ++y)

          {
```

```
                float pcfDepth = texture(shadowMap,
projCoords.xy + offset + vec2(x, y) * (texelSize *
2.0)).r;

                shadow += currentDepth - bias > pcfDepth
? 1.0 : 0.0;

            }

        }

        return shadow / 9.0;

    }
```

Metoda de umbrire de tipul “shadow cube” este o generalizare a metodei de “shadow mapping”, partea de cub venind de la faptul ca lumina produce un ‘shadow map’ in fiecare fata a cubului (sus, jos, stanga, dreapta, fata, spate). Aceasta tehnica este potrivita pentru lumini omnidirectionale (cum ar fi un bec). Acesta functie, pentru a evita calcule matematice grele, si pentru evitarea zgomotului alb, foloseste o metoda popularizata de Jorge Jimene (Interweave Gradient Noise - IGN) pentru a dispersa pixeli intr-un mod ce pare aleator, dar mentine o consistenta vizuala, dar datorita acestei metode, lumina, privita de aproape, tot prezinta un zgomot mic, dar mai bun decat daca s-ar fi aplicat un ‘zgomot alb’. O alta metoda de a face umbrele date de aceasta metoda de “shadow cube” este folosirea unor “sample disks”, cu o distributie Poissoniana, pentru ‘spargerea’ de structuri repetitive din umbra.

Pentru generarea de umbre conform tehnicii shadow cube s-au folosit urmatoarele:

```
    const vec3 sampleDisk[8] = vec3[](

     vec3(0.5238, 0.3411, 0.4110), vec3(-0.2411, 0.5620,
-0.1566),

    vec3(-0.1566, -0.4828, 0.4021), vec3(0.5419, -0.2041,
-0.4411),

    vec3(0.1212, 0.1212, -0.9000), vec3(-0.9000, 0.1212,
0.1212),

    vec3(0.1212, -0.9000, 0.1212), vec3(0.0, 0.0, 0.0)

    );

    float InterleavedGradientNoise(vec2 screenPos)
```

```
  {
      vec3 magic = vec3(0.06711056, 0.00583715,
52.9829189);
      return fract(magic.z * fract(dot(screenPos,
magic.xy)));
  }
  vec2 rotate(vec2 v, float a)
  {
      float s = sin(a);
      float c = cos(a);
      return vec2(v.x * c - v.y * s, v.x * s + v.y *
c);
  }


  float PointShadowCalculation(vec3 fragPos, vec3
lightPos, samplerCube depthCube, float farPlane)
  {
      vec3 fragToLight = fragPos - lightPos;
      float currentDepth = length(fragToLight);
      float spread = (currentDepth / farPlane) * 0.15;
      float rotation =
InterleavedGradientNoise(gl_FragCoord.xy) * 6.2831;
      float shadow = 0.0;
      float bias = 0.05;
      for(int i = 0; i < 8; ++i)
      {
          vec2 rotatedXZ = rotate(sampleDisk[i].xz,
rotation);
          vec3 jitteredDir = vec3(rotatedXZ.x,
sampleDisk[i].y, rotatedXZ.y);
          float closestDepth = texture(depthCube,
fragToLight + jitteredDir * spread).r;
```

```
            closestDepth *= farPlane;
            if(currentDepth - bias > closestDepth)
            {
                shadow += 1.0;
            }
        }
        return shadow / 8.0;
    }
```

Orasul este creat mai intai generand intersectiile (~ semintele), aplicand un offset aleator ("jittered grid") pentru a evita liniile drepte. Dupa aceea este folosit un algoritm de 'grid ray tracing' ("Manhattan Routing") pentru a genera conexiunile (drumurile) dintre aceste intersectii. Acest algoritm preia drumul cel mai scurt in linie dreapta dintre puncte.

Algoritmul este urmatorul:

```
    std::vector<std::vector<CityTileType>>
CityModel::GenerateCity(const std::weak_ptr<Shader>&
shader, int size)
    {
        std::vector<std::vector<CityTileType>>
matrix(size, std::vector<CityTileType>(size,
CityTileType::Skyscraper));
        struct Node
        {
            int x;
            int z;
        };
        std::vector<std::vector<Node>> grid;
        int spacing = (size <= 12) ? 4 : 4;
        for (int z = 1; z < size; z += spacing)
        {
            std::vector<Node> row;
```

```
            for (int x = 1; x < size; x += spacing)
            {
                int ox = (rand() % 2);
                int oz = (rand() % 2);
                row.push_back({ std::clamp(x + ox, 0,
size - 1), std::clamp(z + oz, 0, size - 1) });
            }
            grid.push_back(row);
        }
        auto DrawSimplePath = [&](Node n1, Node n2) {
            int cx = n1.x, cz = n1.z;
            while (cx != n2.x)
            {
                matrix[cz][cx] =
CityTileType::Intersection;
                cx += (n2.x > cx) ? 1 : -1;
            }
            while (cz != n2.z)
            {
                matrix[cz][cx] =
CityTileType::Intersection;
                cz += (n2.z > cz) ? 1 : -1;
            }
            matrix[cz][cx] = CityTileType::Intersection;
        };

        for (size_t r = 0; r < grid.size(); ++r)
        {
            for (size_t c = 0; c < grid[r].size(); ++c)
            {
```

```
                if (c + 1 < grid[r].size())
                {
                    DrawSimplePath(grid[r][c], grid[r][c
+ 1]);
                }
                if (r + 1 < grid.size())
                {
                    DrawSimplePath(grid[r][c], grid[r +
1][c]);
                }
            }
        }
        for (int z = 0; z < size; ++z)
        {
            for (int x = 0; x < size; ++x)
            {
                if (matrix[z][x] ==
CityTileType::Skyscraper)
                {
                    if ((rand() % 100) < 15)
                    {
                        matrix[z][x] =
CityTileType::Park;
                    }
                }
            }
        }
        auto& finalMatrix = matrix;
        auto IsRoad = [&](int r, int c) {
            if (r < 0 || r >= size || c < 0 || c >= size)
```

```
            {
                return false;
            }
            CityTileType t = matrix[r][c];
            return t != CityTileType::Skyscraper && t !=
CityTileType::Park;
        };
        for (int z = 0; z < size; ++z)
        {
            for (int x = 0; x < size; ++x)
            {
                if (matrix[z][x] ==
CityTileType::Skyscraper || matrix[z][x] ==
CityTileType::Park)
                {
                    continue;
                }
                bool N = IsRoad(z - 1, x), S = IsRoad(z +
1, x),
                    E = IsRoad(z, x + 1), W = IsRoad(z, x
- 1);
                int conn = static_cast<int>(N) +
static_cast<int>(S) + static_cast<int>(E) +
static_cast<int>(W);
                if (conn >= 3)    finalMatrix[z][x] =
CityTileType::Intersection;
                else if (N && S)  finalMatrix[z][x] =
CityTileType::StreetVertical;
                else if (E && W)  finalMatrix[z][x] =
CityTileType::StreetHorizontal;
                else if (S && E)  finalMatrix[z][x] =
CityTileType::CornerNE;
```

```
                else if (S && W)  finalMatrix[z][x] =
CityTileType::CornerNW;

                else if (N && E)  finalMatrix[z][x] =
CityTileType::CornerSE;

                else if (N && W)  finalMatrix[z][x] =
CityTileType::CornerSW;

                else              finalMatrix[z][x] =
CityTileType::Intersection;

            }

        }

        BuildModelFromMatrix(shader, finalMatrix);

        Bake(shader);

        return finalMatrix;

    }
```

Pentru algoritmul ‘masinii criminale’ s-a folosit o parcurgere de tip Greedy globala. In esenta, parcurge toate dalele (tile-urile) orasului, si o gaseste pe cea cu cea mai mare distance fata de utilizator, prioritazand mersul in aceeasi directie in care deja se deplaseaza masina, evitand astfel intoarcerile bruste care ar strica experienta simulatorului. Pentru parcurgerea efectiva a drumului pana la acea destinatie, s-a optat pentru algoritmul BFS pentru gasirea drumului, si implicit a nodurilor ce trebuie traversate, fiind un algoritm decent pentru dimensiunea hartii date, desi acesta poate fi inlocuit cu algoritmul A* in dezvoltari ulterioare.

Algoritmii folositi sunt urmatorii:

```
    glm::ivec2
CivilianCarActive::FindFleeTarget(glm::ivec2
playerGrid, const
std::vector<std::vector<CityTileType>>& grid)

    {

        glm::ivec2 bestTile = { 0, 0 };

        float bestScore = -1000000.0f;

        glm::vec2 currentPosGrid =
glm::vec2(WorldToGrid(transform.position));
```

```
      glm::vec2
currentDir(sin(glm::radians(currentAngle)),
cos(glm::radians(currentAngle)));

      for (int y = 0; y < 25; ++y)

      {

          for (int x = 0; x < 25; ++x)

          {

              if (grid[y][x] != CityTileType::Park &&
grid[y][x] != CityTileType::Skyscraper)

              {

                  glm::vec2 tilePos(x, y);

                  float distToPlayer =
glm::distance(tilePos, glm::vec2(playerGrid));

                  glm::vec2 toTileVec = tilePos -
currentPosGrid;

                  float distToTile =
glm::length(toTileVec);

                  if (distToTile < 0.1f)

                  {

                      continue;

                  }

                  glm::vec2 dirToTile = toTileVec /
distToTile;

                  float alignment =
glm::dot(currentDir, dirToTile);

                  float alignmentMultiplier =
(alignment <= 0.0f) ? 0.2f : 1.0f;

                  float score = (distToPlayer *
alignmentMultiplier) + (alignment * 5.0f);

                  if (score > bestScore)

                  {

                      bestScore = score;
```

```
                        bestTile = { x, y };
                    }
                }
            }
        }
        return bestTile;
    }
    std::vector<glm::ivec2>
CivilianCarActive::CalculatePath(glm::ivec2 start,
glm::ivec2 target, const
std::vector<std::vector<CityTileType>>& grid)
    {
        if (start == target) return {};
        std::queue<glm::ivec2> frontier;
        frontier.push(start);
        std::unordered_map<int, glm::ivec2> cameFrom;
        cameFrom[start.x << 16 | start.y] = start;
        bool found = false;
        std::vector<glm::ivec2> neighbors = { {0,1}, {0,-
1}, {1,0}, {-1,0} };
        while (!frontier.empty())
        {
            glm::ivec2 current = frontier.front();
            frontier.pop();
            if (current == target) {
                found = true;
                break;
            }
            for (const auto& offset : neighbors)
            {
```

```
                glm::ivec2 next = current + offset;
                if (next.x >= 0 && next.x < 25 && next.y
>= 0 && next.y < 25)
                {
                    int nextKey = next.x << 16 | next.y;
                    if ((grid[next.y][next.x] !=
CityTileType::Park && grid[next.y][next.x] !=
CityTileType::Skyscraper) && cameFrom.find(nextKey) ==
cameFrom.end())
                    {
                        frontier.push(next);
                        cameFrom[nextKey] = current;
                    }
                }
            }
        }
        std::vector<glm::ivec2> finalPath;
        if (found)
        {
            glm::ivec2 curr = target;
            while (curr != start)
            {
                finalPath.push_back(curr);
                curr = cameFrom[curr.x << 16 | curr.y];
            }
            std::reverse(finalPath.begin(),
finalPath.end());
        }
        return finalPath;
    }
```

Efectul de ‘xray’ cand ‘masina criminala’ se departeaza este realizat print-un shader simplu, cel de fragment desenand o singura culoare data cu o anumita opacitate. Opacitatea este calculata in functie de distanta. Pentru ca acei pixeli sa fie vizualizati deasupra celorlati pixeli, sa folosit functia glDepthFunc(GL_GREATER).

```
      auto& xrayS =
shaders[static_cast<size_t>(ShaderType::Xray)];

          float distance = glm::distance(mainCar-
>GetPosition(), chaseCar->GetPosition());

          if (distance > 50.0f)

          {

              xrayS->UseShader();

              glEnable(GL_BLEND);

              glBlendFunc(GL_SRC_ALPHA,
GL_ONE_MINUS_SRC_ALPHA);

              glDepthMask(GL_FALSE);

              glDepthFunc(GL_GREATER);

              glUniformMatrix4fv(xrayS-
>GetUniformLocation("projViewMatrix"), 1, GL_FALSE,
&projViewMatrix[0][0]);

              float alpha = glm::clamp((distance -
15.0f) / 100.0f, 0.2f, 0.7f);

              glUniform3f(xrayS-
>GetUniformLocation("glowColor"), 1.0f, 0.4f, 0.0f);

              glUniform1f(xrayS-
>GetUniformLocation("opacity"), alpha);

              chaseCar->Draw(xrayS);

              glDepthFunc(GL_LESS);

              glDepthMask(GL_TRUE);

              glDisable(GL_BLEND);

          }
```

Pentru a ne folosi de texturare, pentru opacitatea ferestrelor masinilor sa folosit un asa zis “programming hack”. In fragment shader-ul phong, culoarea

este doar un vector de 3 elemente (RGB), dar textura este un (RGBA), asa ca transparenta ferestrelor sa realizat prin setarea texturii acestora la o textura de 1x1 pixeli cu o culoare si o transparenta data.

```
  {
  if (!data->windowTex)
  {
      data->windowTex = Texture::CreateSolidColor(
          static_cast<unsigned char>(color.r * 255),
          static_cast<unsigned char>(color.g * 255),
          static_cast<unsigned char>(color.b * 255),
          alpha,
          "texture_diffuse"
      );
  }
  std::unique_ptr<Mesh> mesh = nullptr;

  switch (type)
  {
  case WindowType::Triangle:
  {
      mesh = std::make_unique<WedgeMesh>(shader,
Transform{ .position = pos, .rotation = rot, .scale =
scale }, color);
      break;
  }
  case WindowType::Flat:
  default:
  {
      mesh = std::make_unique<CubeMesh>(shader,
Transform{ .position = pos, .rotation = rot, .scale =
scale }, color);
```

```
        break;
    }
    }
    CivilianCarPart& part = CreatePart(std::move(mesh),
CivilianCarMeshType::Window,
CivilianCarPartIndex::None, pos);
    }
    Texture::Texture(uint8_t* data, int width, int
height, int components, const std::string& uniformName)
:
        uniformName(uniformName)
    {
        glGenTextures(1, &ID);
        glBindTexture(GL_TEXTURE_2D, ID);
        glPixelStorei(GL_UNPACK_ALIGNMENT, 1);
        GLenum format = (components == 4) ? GL_RGBA :
GL_RGB;
        glTexImage2D(GL_TEXTURE_2D, 0, format, width,
height, 0, format, GL_UNSIGNED_BYTE, data);
        glTexParameteri(GL_TEXTURE_2D,
GL_TEXTURE_MIN_FILTER, GL_LINEAR);
        glTexParameteri(GL_TEXTURE_2D,
GL_TEXTURE_MAG_FILTER, GL_LINEAR);
    }
    std::shared_ptr<Texture>
Texture::CreateSolidColor(uint8_t r, uint8_t g, uint8_t
b, uint8_t a, const std::string& uniformName)
    {
        uint8_t data[] = { r, g, b, a };
        return std::make_shared<Texture>(data, 1, 1, 4,
uniformName);
    }
```

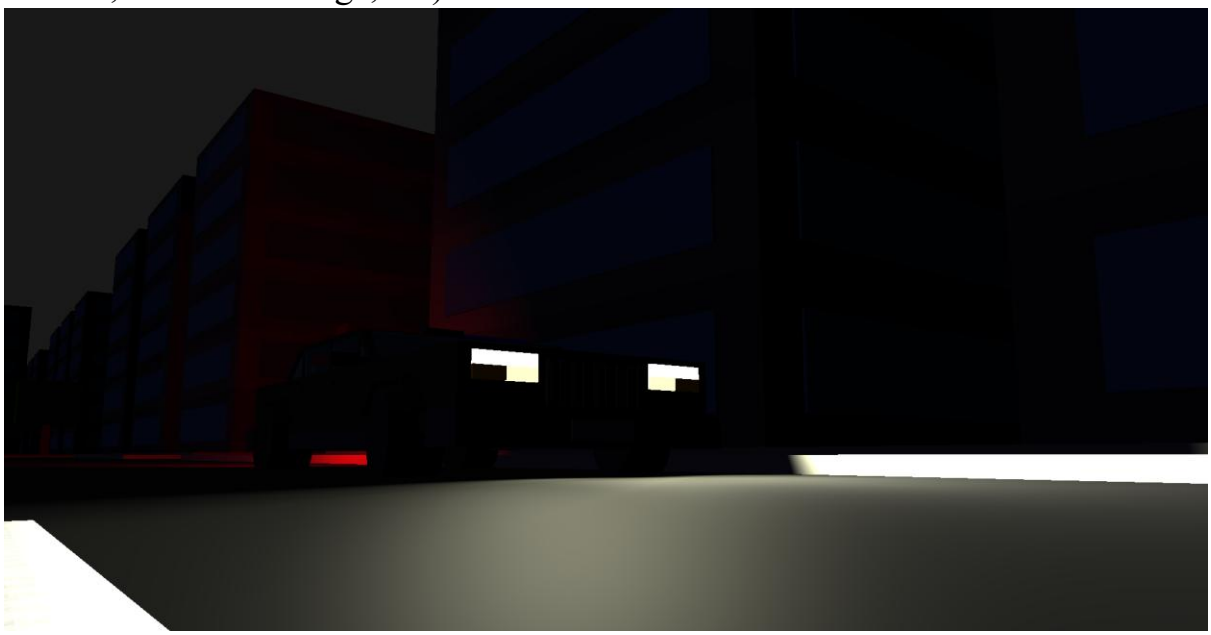
# III. Rezultate

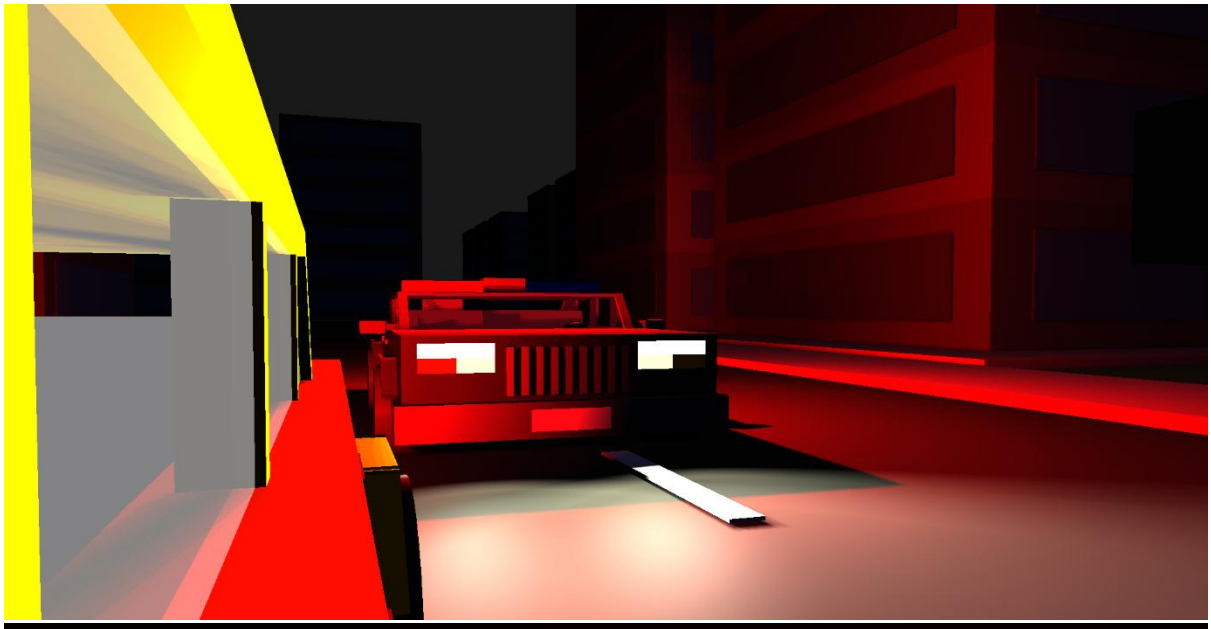
In urma multor cicluri de dezvoltare, s-a ajuns la un demo acceptabil al simulatorului. Acesta include masina de politie controlata de utilizator, posibilitatea de a sting sau aprinde luminiile al respectivei masini, orasul generat, ‘masinii criminale’ si algoritmii aferenti, metodelor de iluminare si umbrire mentionate mai sus. Pe langa acestea, s-a implementat si o modalitate de a instantia oricate masini de politie statice, desi aceasta functionalitate nu este folosite in simulator inca.

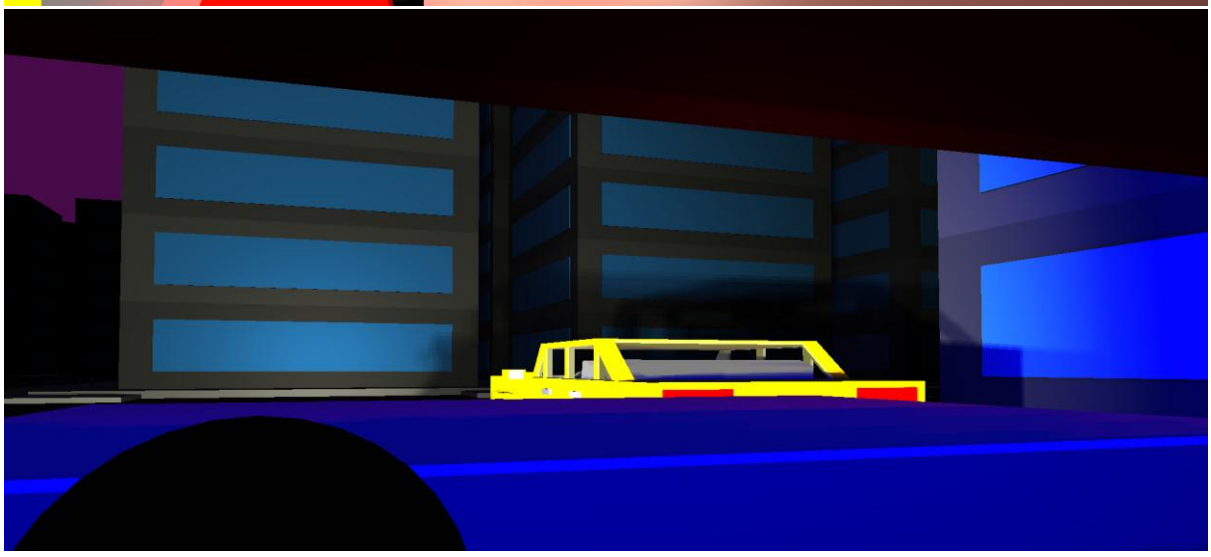
Efectele de umbra sunt satisfacatoare pentru demo-ul dat, nu avand efect de “moire” (“shadow acne”), si cu o decuplare de obiect (“peter panning”) minima.

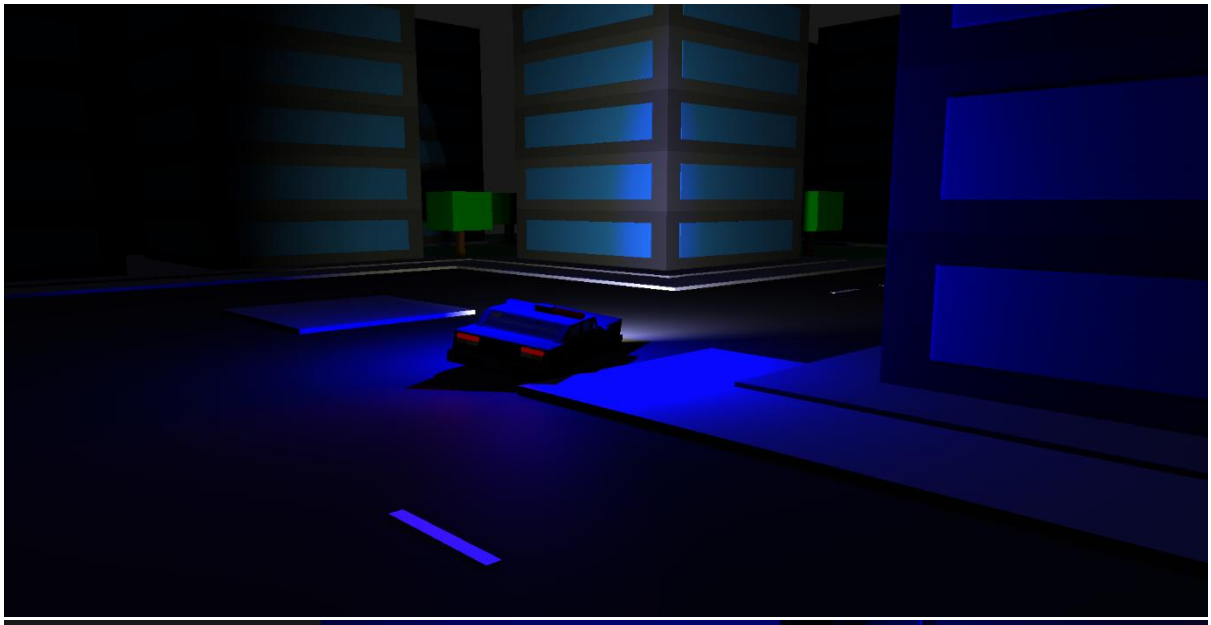
Efectul de X-RAY este implementat corect, dar poate disparea daca ‘masina criminala’ trece de ‘far plane’-ul camerei (~ 200 unitati).

O alta functionalitate deosebit de importanta este tratarea reala a acceleratiei, vitezei si deceleratiei masinii, desi nu la un nivel foarte avansat, are un comportament natural si de asteptat, dar in acest sens, schimbarile de vitezi si directia afecteaza rotatia masinii intr-un mod natural (datorita inertiei, fortei centrifuge, etc)









# IV. Concluzii

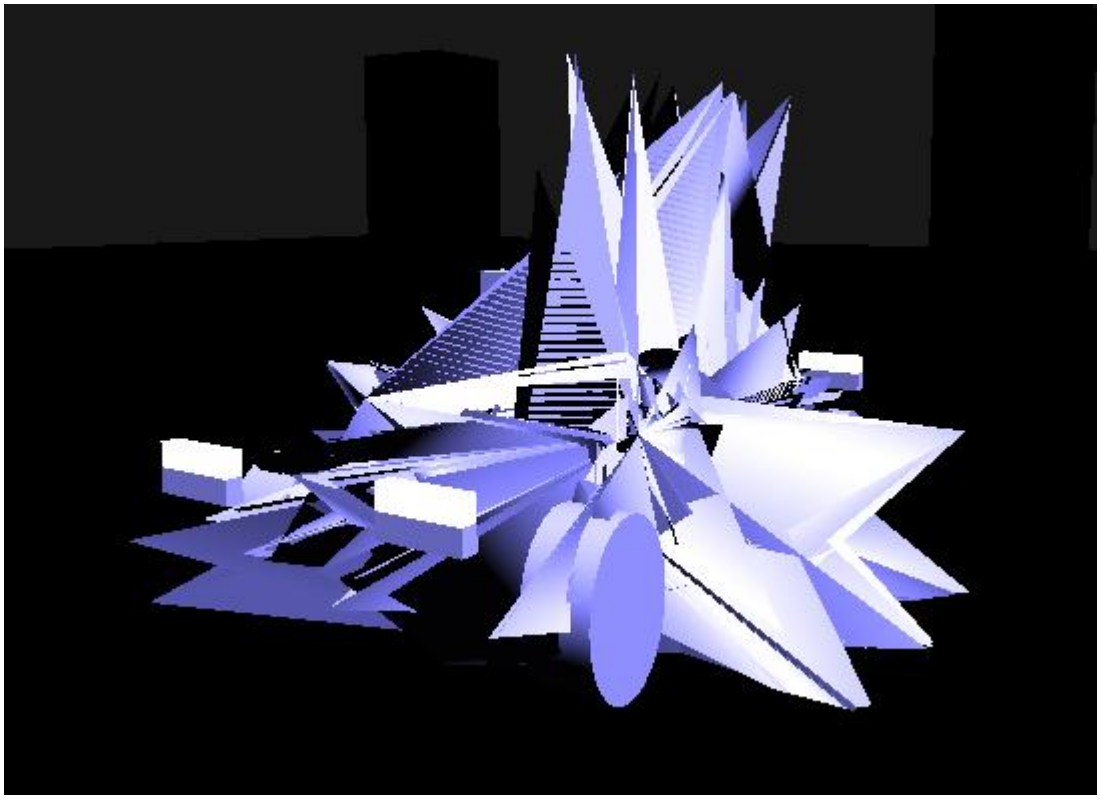
In acest proiect sa implementat un simulator al unei masini de politie, intr-un oras, in timpul noptii. Utilizatorul poate conduce masina si controla luminile. Simulatorul s-a concentrat pe partea vizuala, avand implementate modelul de iluminare phong, precum si modelele de umbrire “shadow map” si “shadow cube”.

Eliminarea efectului de ‘moire’ in timp ce se incarca sa se minimizeze efectul de decuplare a umbrelor a reprezentat un punct de dificultate marita. Un alt punct dificil a fost gasirea unei balante intre fidelitate grafica si performanta. Performanta curenta a simulatorului este acceptabila, dar pentru

orice alte dezvoltari ulterioare, necesita optimizari majore, desi fidelitatea grafica curenta este la un nivel satisfacator.

O directie de dezvoltare este restructurarea responsabilitatilor claselor mai mari (Application.h, PoliceCarModel.h, CivilianCarActive.h), prin spargerea acestora in alte clase. Pe partea de mecanici ale simulatorului se pot adauga coliziuni cu medium, dar si un sistem de fizica mai avansat, ceea ce ar face posibila adaugarea functionalitatii de suspensii ale masinilor.

Contributia personala a fost proiectarea cat mai balansata intre viteza si corectitudine, precum si separarea responsabilitatilor, dar si idea de proiect.



# V. Referinte Bibliografice


Interweaved Gradient Noise - https://blog.demofox.org/2022/01/01/interleaved-gradient-noise-a-different-kind-of-low-discrepancy-sequence/

Learn Open GL (model de iluminare, tehnicile de shadow mapping, shadow cube, texturare, camera, meshes, shaders) - https://learnopengl.com/

Rule of Three / Five / Zero - https://en.cppreference.com/cpp/language/rule_of_three

GLFW - https://github.com/glfw/glfw

GLAD - https://github.com/dav1dde/glad

RAII - https://en.cppreference.com/cpp/language/raii

OpenGL wiki - https://wikis.khronos.org/opengl/Main_Page

How to only get one key press on GLFW - https://stackoverflow.com/questions/51873906/is-there-a-way-to-process-only-one-input-event-after-a-key-is-pressed-using-glfw